# Nios® V/g Processor Tightly Coupled Memory (TCM) Design

## Agilex™ 7 FPGA

# Contents

# 1. Theory of Operation

Nios V/g Processor-based design example with Tightly Coupled Memory (TCM) on Agilex™ 7 FPGA.

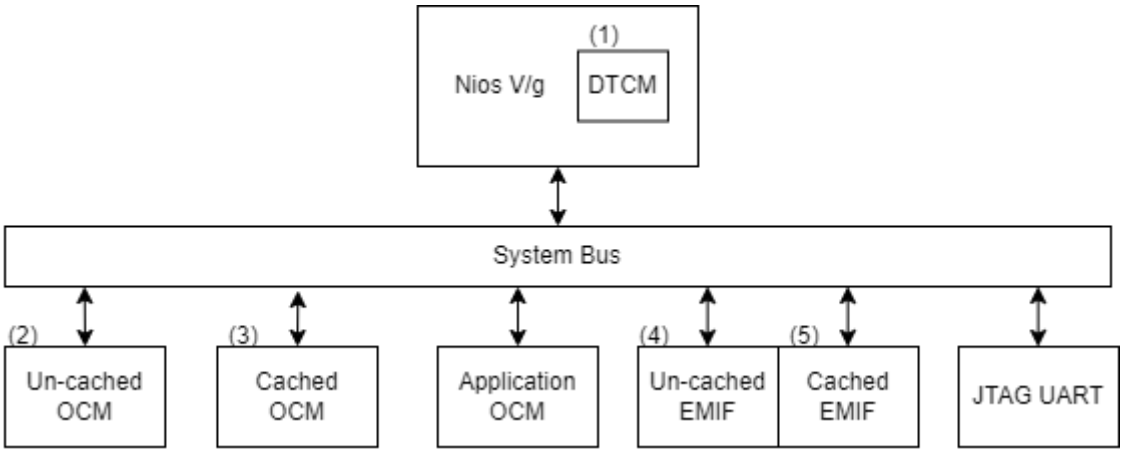
This example design is about how to use tightly coupled memory in Nios V/g processor. The example application measures the memory access speed of different memories connected to the processor, such as TCM, on-chip memory and external memory interface (EMIF). In addition to that, the application showcases the speedup between cached and un-cached memories. This document guides you through the process of building a Nios V system with tightly coupled memory.

The interested memories are:

1. Tightly Coupled Memory
2. Un-cached On-Chip RAM in Peripheral Region
3. Cached On-Chip RAM
4. Un-cached EMIF in Peripheral Region
5. Cached EMIF

Measuring memory speed in a Nios V processor system is crucial because it represents the overall performance and efficiency of the system. Faster memory access leads to quicker data retrieval and better application performance. It helps identify bottlenecks, ensuring that the memory subsystem is well-matched with the processor. This measurement is essential for optimizing system design, benchmarking against other configurations, and meeting the timing requirements of real-time applications. Additionally, understanding memory speed aids in making trade-offs between performance and power efficiency, troubleshooting issues, and planning for future upgrades and scalability.

**Figure 1-1. Block Diagram**



### 1.1. IP Cores

The following IPs are used in this design.

- Nios V/g soft processor core
- Three On Chip RAM-II
  - OCRAM_0 for application execution only
  - Cached OCRAM_1
  - Un-cached OCRAM_3
- EMIF IP for memory access test
  - 1st half of the EMIF is cached.
  - 2nd half of the EMIF is un-cached.
- JTAG UART
- Performance Counter IP
- System ID

## 2. Directory Structure

The directory structure is explained below:

Root directory: `agf014eb-si-devkit/niosv_g/tcm_mem_test`

| Directories | Content |
|---|---|
| docs | Document capturing the details of the example design |
| img | Block diagram for the example designs |
| ready_to_test | Prebuilt binaries (FPGA `.sof` file and Application `.elf` file) which can be programmed on the board and tested |
| sources<br>- hw<br>- scripts<br>- sw/app | Files needed to create the design<br>- Necessary hardware files (`.sv`, `.v`, `.ip`) of the design<br>- This folder consists of scripts to build the design<br>- This folder contains software application files |

## 3. Quick Start Guide

You may try the example on the target development kit using the prebuilt binaries. The respective FPGA `sof` and Application `elf` files are found in `ready_to_test` folder.
Refer to Chapter 6 Programming the Design for the steps to program these files.

# 4. Generating the Design

The implementation of Nios® V processor system requires a hardware design developed using the Quartus® Prime software. After that, it is required to develop the software design that complements the processor system.

## 4.1. Building Hardware Design

Begin designing the system hardware by instantiating the Nios® V processor and its peripherals into Platform Designer. After configuring the system assignments and constraints, complete the hardware design by performing a successful compilation.

We will be using `sources/scripts/build_sof.py` to develop the hardware design.

This script will create a new platform design, add the IPs into the design, makes the connections, compiles the design and generates the .sof file.

Steps to execute the script are as follows:

1. Invoke the Nios V Command Shell in the terminal
2. Run the following command in the terminal from top level project directory: `> quartus_py ./scripts/build_sof.py`
3. The Quartus tool will compile the design and generate the output files into `<Root Directory>/sources/hw/output_files`

## 4.2. Building Software Design

After the processor system is ready, you may begin building the software design. It consists of the following steps:

1. Create a board support package (BSP) project.
2. Create a Nios® V processor application project with source code.
3. Build the application.
4. Convert the application .elf file into .hex file for memory initialization.

To create software application, run the following commands in the terminal:

```
> niosv-bsp -c --quartus-project=hw/<>.qpf --qsys=hw/<>.qsys --type=hal --script=sw/bsp_linker_script.tcl sw/bsp/settings.bsp

> niosv-app --bsp-dir=sw/bsp --app-dir=sw/app --srcs=sw/app/tcm.c

> niosv-shell

> cmake -S ./sw/app -B sw/app/build -G "Unix Makefiles"

> make -C sw/app/build

> elf2hex sw/app/build/app.elf -b 0x3FFF -w 32 -e 0x7FFF sw/app/build/itcm.hex -r4
```

Note: It is recommended to clean the `app/build` project before regenerating `elf` (cmake and make).

## 6. Programming the Design

Use the Quartus® Prime Programmer tool and the Ashling* RiscFree* IDE to program the Nios® V processor-based system into the FPGA and to run your application.

Program the generated `sof`, then download the `elf` file on the board according to the command below.

```
> quartus_pgm --cable=1 -m jtag -o 'p;ready_to_test/top.sof'

#----- Download the elf file on the board ----#

> niosv-download -g ready_to_test/app.elf -c 1

#----- Verify the output on the terminal by using the following command in the terminal -----#

> juart-terminal -c 1 -i 0
```

Note: Reduce the JTAG clock frequency to 6MHz before programming the application `.elf` file on the board using the command: `jtagconfig --setparam 1 JtagClock 6M`

# 7. Expected Results

The following is the output as observed on the JTAG UART terminal. The output is analogous to the logic from the application code. Users should be able to observe the similar output on their terminal/setup.

It is a printout of the statistics that shows the higher speeds obtained by leveraging tightly coupled memories. Note that the number of clock cycles for tightly coupled memory is very similar to that of the cached memory. The result demonstrates that tightly coupled memories allow fixed low latency read access to executable code as well as fixed low-latency read, write, or read and write access to data.

Note: The timing numbers output varies between Nios development boards

**Figure 1-2. Expected Result**

```
Tightly Coupled Memory vs. Cache Example Real-Time Measurements:

AGILEX7 (ag_qsys).
Checksum Times:                         All 5 memories match.
Tightly coupled memory:     36.13 microseconds,     3613 clocks.
On-chip memory:             52.13 microseconds,     5213 clocks.
On-chip memory (cached):    36.13 microseconds,     3613 clocks.
EMIF memory:               100.92 microseconds,    10092 clocks.
EMIF memory (cached):       36.86 microseconds,     3686 clocks.

Checksum loop measured iteration:  3.
Checksum value:                   34465 total.
Checksum block size:                320 words (32 bits).
CPU Frequency:                 100.0000 Mhz.
TCM Test Passed
```